VIETNAM NATIONAL UNIVERSITY, HANOI

UNIVERSITY OF ENGINEERING AND TECHNOLOGY

Hoang Duc Bach

DESIGN AND IMPLEMENTATION OF AN INTENT-BASED META DEX AGGREGATOR WITH COINCIDENCE-OF-WANTS MATCHING ON SUI BLOCKCHAIN

BACHELOR THESIS

Major: Information Technology

HA NOI - 2026

VIETNAM NATIONAL UNIVERSITY, HANOI

UNIVERSITY OF ENGINEERING AND TECHNOLOGY

Hoang Duc Bach

DESIGN AND IMPLEMENTATION OF AN INTENT-BASED META DEX AGGREGATOR WITH COINCIDENCE-OF-WANTS MATCHING ON SUI BLOCKCHAIN

Major: Information Technology

Supervisor: Dr. Nguyen Thi Hau

HA NOI - 2026

Acknowledgments

I would like to express my sincere gratitude to my supervisor, Dr. Nguyen Thi Hau, for her guidance, valuable feedback, and continuous support throughout this research. Her insights and suggestions greatly helped improve the quality of this thesis.

I also gratefully acknowledge the support of the Sui and Deepbook teams for their generous provision of testnet token funding, which made the practical implementation of this research possible. I extend equal thanks to the broader Sui community for their open and collaborative spirit — their technical discussions, shared resources, and willingness to engage with emerging research have enriched this work in ways that extend well beyond what any single reference could capture.

I am equally thankful to my family, whose steady encouragement and unwavering belief in me sustained me through the most demanding stages of this work.

This thesis is, in many ways, a reflection of the collective generosity of those who supported me — academically and personally. To each of them, I am sincerely grateful.

Declaration

I, Hoang Duc Bach, declare that this thesis is my own original work, composed independently under the supervision of Dr. Nguyen Thi Hau. It has not been submitted, in whole or in part, for any other degree or qualification at this or any other institution.

All sources consulted in the preparation of this thesis have been fully acknowledged and cited in accordance with academic convention. I accept full responsibility for the integrity of this work and for any consequence arising from a failure to meet this declaration, in accordance with the regulations of the University of Engineering and Technology, Vietnam National University.

Undergraduate student,

Hoang Duc Bach

Abstract

As the decentralized finance (DeFi) landscape continues to evolve, slippage and Maximal Extractable Value (MEV) remain significant hurdles for users on Ethereum and EVM-compatible chains. While CoW DEX solutions have successfully mitigated these issues in the EVM space, the Sui network presents a unique environment where its Object-Centric model inherently provides robust protection against certain forms of MEV. Nevertheless, slippage continues to be a persistent challenge for decentralized exchanges (DEXs) on the Sui network.

This thesis focuses on the analysis, architectural design, and development of a CoW DEX on Sui, leveraging the network's object-oriented paradigm and inspired by the core architecture of the Ethereum CoW Protocol. The research proposes a novel auction mechanism incorporating a Commit-Reveal phase and utilizes the Hot Potato Pattern to ensure atomicity, fairness, and the prevention of solver fraud. The system aims to establish an intent-based DEX aggregator that delivers optimal pricing and zero slippage for coincident orders (Coincidence of Wants), while effectively harnessing Sui's core liquidity layers, specifically Deepbook. This work contributes a scalable and secure trading framework that enhances transparency and capital efficiency within the Sui ecosystem.

Keywords: *Blockchain, Decentralized Finance, Coincidence of Wants, Slippage, Maximal Extractable Value, Intent-based, Object-Centric, Sui, Ethereum, Deepbook, CoW Protocol.*

Contents

Acknowledgments

Declaration

Abstract

List of Figures

List of Tables

1 Introduction **1**

1.1 Motivation 1

1.2 Problem Statement 5

2 Related Works **8**

3 Intent-Based Model and CoW Mechanism on Sui **9**

3.1 Overview of CoW Protocol 9

3.1.1 Architecture and Design Philosophy 9

3.1.2 Order Flow 9

3.1.3 Enforcement Architecture 10

3.1.4 Fair Combinatorial Batch Auction 11

3.1.5 Forms of Coincidence of Wants 11

3.1.6 The Solver Whitelist Mechanism 12

3.2 CoW Protocol on Ethereum versus the Proposed Design on Sui 12

3.2.1 Structural Limitations of the Account-Based Model 12

3.2.2 Design Opportunities of Sui's Object Model 13

3.3 Proposed Model: On-Chain Intent-Based Batch Auction on Sui 14

3.3.1 Intent as an On-Chain Shared Object 14
3.3.2 On-Chain Batch Auction Lifecycle 14
3.4 On-Chain Solver Selection via Commit-Reveal 15
3.4.1 Why Commit-Reveal Rather Than Open Bid 15
3.4.2 Game-Theoretic Analysis of the Bond Mechanism 16
3.4.3 On-Chain Penalty Conditions 16
3.5 Type-Level Atomicity in Move 17
3.5.1 EVM Revert versus Type-Level Correctness 17
3.5.2 The Hot Potato Pattern in Move 17
3.6 Matching Algorithm and Surplus Theory 18
3.6.1 Surplus Definition and Winner Selection Metric 18
3.6.2 CoW Validity Conditions 19
3.6.3 On-Chain Price Floor as a Substitute for EBBO 19

References **20**

Appendices **22**

A This is title of appendix A **22**

B This is title of appendix B **23**

List of Figures

1.1 Global DeFi and DEX ecosystem TVL trends from 2018 to early 2026, demonstrating sustained growth despite market corrections. 2
1.2 CoW Protocol batch history on Ethereum, showing batch frequency and volume characteristics demonstrating real-world matching patterns and protocol activity. 3
1.3 Sui DeFi ecosystem composition, showing the dominance of AMMs and aggregators as of Q2 2025, with no existing CoW-style protocol. 4

List of Tables

Chapter 1

Introduction

1.1 Motivation

The emergence of blockchain technology has catalyzed a fundamental transformation in financial infrastructure, giving rise to a new class of decentralized financial systems that operate without traditional intermediaries. As of early 2026, global DeFi Total Value Locked (TVL) stands at approximately $125 billion — a figure that, despite a seasonal correction from the mid-2025 peak of $2.6 billion on Sui alone, reflects the enduring adoption and long-term growth trajectory of on-chain finance. Within this ecosystem, decentralized exchanges (DEXs) have emerged as the primary venue for permissionless asset trading, sustaining an average global daily volume of approximately $19 billion throughout 2025 and into 2026, with Sui-native DEXs contributing approximately $250 million daily [1].

As highlighted in the OKX State of DEXs 2025 Report, the central promise of decentralized trading is the delivery of low slippage and best-price execution. Since the inception of DEXs in 2018, successive architectural generations have pursued this goal through distinct mechanisms. Automated Market Makers (AMMs) democratized liquidity provision but introduced price impact proportional to trade size. Order book DEXs improved price precision but required active market participation and remained susceptible to liquidity fragmentation. DEX Aggregators improved execution by routing across multiple venues, yet remained fundamentally constrained by the depth of underlying pools. Hybrid models combine these approaches at the cost of architectural complexity. Each generation represents an incremental improvement; none fully

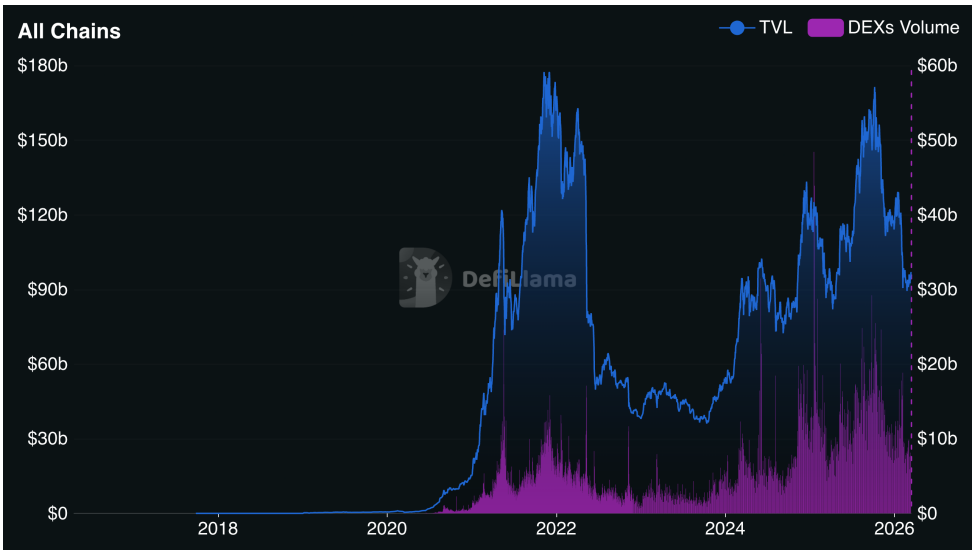

Figure 1.1: Global DeFi and DEX TVL trends (2018–2026).

resolves the slippage problem under fragmented liquidity conditions.

It is in this context that Maximal Extractable Value (MEV) emerges as a critical phenomenon. MEV refers to the profit that block producers and sophisticated market participants can extract by strategically reordering, including, or excluding transactions within a block prior to finalization [2]. This is made possible by the public visibility of pending transactions in the mempool: before a trade is confirmed, its parameters are observable by any network participant. Searchers — automated agents monitoring the mempool — exploit this transparency to execute three primary forms of MEV extraction. In **front-running**, a searcher identifies a profitable pending trade and submits an identical transaction with a higher priority fee, ensuring execution ahead of the original user and capturing the resulting price movement. In a **sandwich attack**, the attacker places a buy order immediately before the target transaction and a sell order immediately after, artificially inflating the execution price for the victim while profiting from the induced price delta. In **back-running**, the attacker inserts a transaction directly following a price-moving event to capture the resulting arbitrage opportunity before the market equilibrates [2].

The implications of MEV extend beyond individual transaction losses. At scale, MEV introduces systemic inefficiencies: it increases effective transaction costs for ordinary users, distorts price formation on DEXs, and creates an asymmetric information advantage for sophisticated actors with access to specialized infrastructure. These effects collectively undermine the fairness and accessibility that decentralized finance

purports to guarantee [2].

In response to MEV's growing impact, multiple mitigation approaches have emerged. Flashbots introduced private transaction relaying to reduce mempool exposure; ZeroMEV developed analytics tooling to quantify and attribute extracted value; and CoW Protocol introduced a structurally distinct execution model. Proposed in 2021, CoW Protocol's **Coincidence of Wants** mechanism batches user trade intents and searches for peer-to-peer matches — settling directly between counterparties before routing residual volume to external liquidity sources. When a user intending to swap token A for token B is matched with a counterparty intending the inverse trade, both orders are settled at the reference market price with zero AMM fee and zero price impact. This batch auction model simultaneously eliminates MEV exposure, reduces slippage, and recovers surplus for users rather than extractors. According to on-chain data from Dune Analytics, even on Ethereum the average CoW batch contains approximately 1.5 intents, illustrating that peer-to-peer matching remains probabilistic and volume-dependent [3].

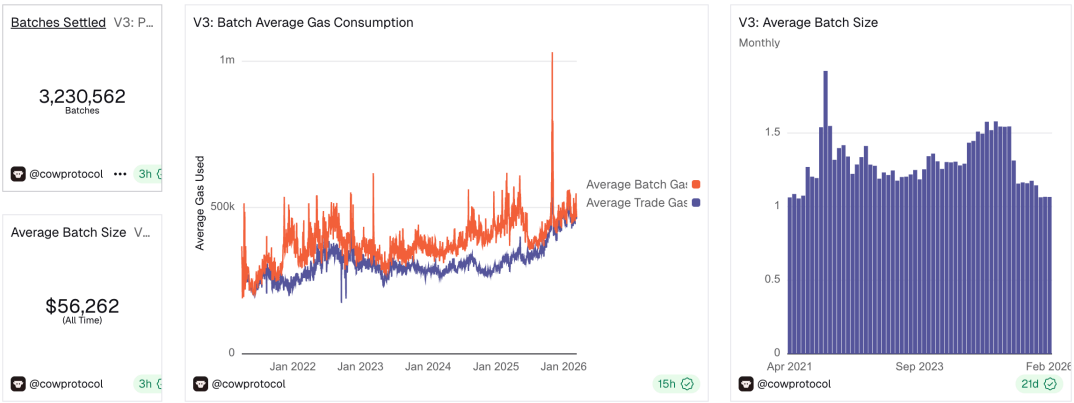

Figure 1.2: CoW Protocol batch history on Ethereum.

On Sui, MEV is addressed through a different set of architectural mechanisms — including priority gas auctions (PGAs) and consensus amplification via SIP-45 — with the explicit design principle that user transaction protection takes priority over extracted value [4]. As a result, the MEV problem that primarily motivates CoW on Ethereum is partially mitigated by Sui's protocol design. Nevertheless, the CoW model retains a powerful and distinct advantage: the ability to achieve **zero slippage** when sufficient matching volume exists. Furthermore, a CoW-style protocol on Sui does not compete with or displace existing DEXs. Rather, it functions as a **meta-DEX**

aggregator that discovers and exploits better pricing across Cetus, Aftermath, DeepBook, and other Sui protocols — reducing liquidity fragmentation rather than adding to it. As confirmed by the Sui Q2 2025 DeFi Roundup [1], AMMs and aggregators currently dominate Sui DeFi volume, and no major CoW-style batch auction protocol exists on the network, representing a clear gap in the ecosystem.

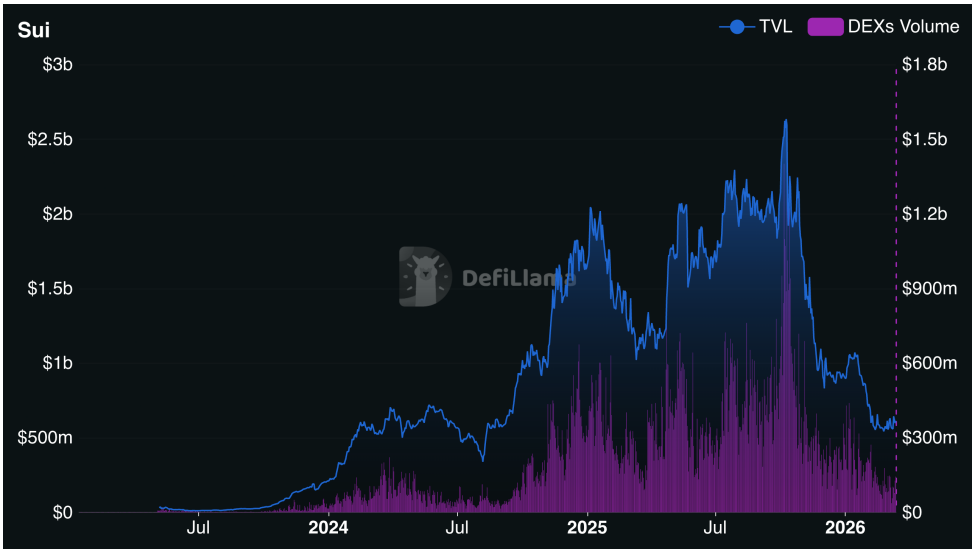

Figure 1.3: Sui DeFi ecosystem: AMMs and aggregators dominance (Q2 2025).

Understanding the behavior, scale, and distribution of MEV — as well as the effectiveness of countermeasures such as CoW-style batch auctions — has become essential for designing fair and efficient decentralized trading systems. However, existing implementations of CoW Protocol on Ethereum rely on off-chain infrastructure for solver coordination, intent relay, and price verification, reintroducing trusted intermediaries whose failure modes have been demonstrated in practice: a solver network exploit resulted in direct fund losses, and a separate incident in which the off-chain permission model caused a user to lose nearly $50 million in a misrouted trade — both illustrating the systemic risk of off-chain trust assumptions [5].

This thesis addresses the gap between the theoretical promise of the CoW model and its current implementation constraints. Leveraging Sui's object-centric architecture — which provides native primitives for shared object ownership, programmable transaction blocks, and sub-second finality — this work designs and implements a trustless CoW batch auction protocol in which the entire lifecycle, from intent locking through solver competition to atomic on-chain settlement, is enforced by smart contract logic with no off-chain coordinator. The result is a meta-DEX aggregator that

complements Sui's existing liquidity ecosystem, reduces fragmentation, and provides users with best-execution guarantees backed by on-chain price verification.

1.2 Problem Statement

Although the Coincidence of Wants model has demonstrated its effectiveness in optimizing trade execution and reducing slippage on Ethereum, its application within the Sui ecosystem remains largely unexplored. This thesis identifies four core problems that motivate the proposed design.

P1 — Absence of a CoW-based Protocol on Sui

Despite the significant growth of Sui's DeFi ecosystem — with total DEX volume exceeding $33 billion in Q2 2025 alone [1] — no major protocol operating on the CoW batch auction model currently exists on the network. Existing protocols, including AMMs such as Cetus, Bluefin, and Aftermath, and aggregators such as 7k and FlowX, operate independently and lack a coordination layer capable of identifying and exploiting coincidence-of-wants opportunities across concurrent trade intents. As a consequence, liquidity remains fragmented across isolated pools, and users continue to incur unnecessary slippage even in market conditions where sufficient opposing intent volume exists. The absence of a CoW-style meta-DEX aggregator means the Sui ecosystem foregoes the possibility of zero-slippage execution in eligible trade conditions, while leaving unexploited the natural alignment between Sui's object-centric architecture and the intent-based trading model.

P2 — Off-chain Trust Assumptions and Permission Risk in Existing CoW Implementations

The architecture of CoW Protocol on Ethereum relies substantially on off-chain infrastructure for core protocol functions, including intent relaying, solver coordination, and price verification. This design reintroduces dependence on trusted intermediaries — a dependency that decentralized finance is fundamentally intended to eliminate. The practical consequences of this trust assumption have been demonstrated through two documented incidents. First, an exploit targeting CoW Protocol's solver network resulted in direct user fund losses [6]. Second, a separate incident in which the off-chain permission and approval model caused a user to lose approximately $50

million in a misrouted trade — receiving only a $600,000 fee refund as compensation [5]. Both cases illustrate that when critical execution decisions are processed off-chain, users cannot rely on smart contract guarantees to protect their assets. A genuinely trustless system requires that the entire transaction lifecycle — intent locking, solver competition, price verification, and settlement — be enforced and verifiable on-chain.

P3 — High Operational Cost and Solver Capital Inefficiency

On Ethereum-based deployments, high gas costs impose two structural constraints on the efficiency of the CoW model. First, per-transaction costs force solvers to accumulate a sufficient number of intents before a batch becomes economically viable, resulting in extended user wait times. In practice, CoW Protocol on Ethereum processes batches approximately every ten minutes [3], during which users have no execution guarantee. Second, solvers must maintain substantial capital reserves to fund settlement for complex batches, creating high barriers to entry and limiting competition within the solver network. On Sui, significantly lower gas fees and sub-second transaction finality open the possibility of high-frequency batch processing — enabling what may be characterized as micro-batching. Furthermore, Sui's Programmable Transaction Block (PTB) mechanism allows solvers to source on-chain liquidity from DeepBook within the same atomic transaction as settlement, reducing capital requirements to near zero and lowering the barrier for solver participation.

P4 — Liquidity Fragmentation and the Absence of an Optimal Routing Layer

Even in the absence of direct peer-to-peer matching opportunities, users on Sui face the challenge of identifying optimal execution paths across fragmented liquidity venues. Existing aggregators address this partially by routing individual orders across multiple pools, but they operate reactively — processing each order in isolation as it arrives, without the ability to combine concurrent intents and exploit potential CoW matches within a shared time window. As a result, a meaningful portion of trades continues to incur unnecessary slippage by being routed through AMM pools when a directly opposing intent, submitted within the same batch window, could have fulfilled the order at a better price. A meta-aggregator built on the batch auction model can address both dimensions simultaneously: maximizing the probability of peer-to-peer CoW matching and guaranteeing optimal AMM routing for residual unmatched volume.

Chapter 2

Related Works

Chapter 3

Intent-Based Model and CoW Mechanism on Sui

3.1 Overview of CoW Protocol

3.1.1 Architecture and Design Philosophy

CoW Protocol functions as a coordination layer that aggregates liquidity across multiple sources, including on-chain AMMs and order book DEXs as well as off-chain market makers and CEX inventory [7]. Unlike traditional DEXs, where users bear responsibility for selecting execution routes and managing associated risks, CoW Protocol instead delegates these decisions to a competitive network of solvers. The protocol enforces constraints designed to protect user outcomes.

The protocol relies on a network of solvers that discover execution opportunities by scanning available liquidity across multiple venues. This includes on-chain AMMs, DEX aggregators, private market makers, and peer-to-peer matching opportunities between intents within the same batch. Solvers compete by proposing solutions that maximize total surplus across all batch participants, with the winning solver compensated in COW tokens.

3.1.2 Order Flow

A trade on CoW Protocol involves several phases as documented by the protocol authors [7]. Users first submit trade intents off-chain by signing messages that specify

desired assets and minimum acceptable prices. At this stage, user assets are not locked. Instead, users must have previously authorized CoW Protocol's vault relayer via ERC-20 `approve` [8], granting it permission to move tokens on their behalf.

The protocol then aggregates intents into a Fair Combinatorial Batch Auction. After a batch closes, no new intents can be added, and the solver competition begins. Solvers have approximately 36 seconds (three blocks on Ethereum Mainnet) to propose solutions via an off-chain coordination component called the Autopilot. Because the auction operates entirely off-chain and remains opaque to all participants, solvers cannot observe each other's proposals before submission.

Finally, the winning solver submits a settlement transaction on-chain. The `GPv2Settlement` smart contract verifies each intent's signature and enforces the minimum price constraint. If execution prices fall below user-specified minimums, the transaction reverts immediately.

3.1.3 Enforcement Architecture

CoW Protocol distributes enforcement responsibilities across three distinct layers, each with specific scope.

The smart contract layer enforces two core constraints: limit prices and solver authorization. The protocol verifies that execution respects user-specified minimum thresholds and restricts settlement submissions to whitelisted solver addresses.

On-chain enforcement alone cannot handle the full complexity of the auction process. The off-chain Autopilot layer computes solver scores, validates pricing, and selects winners [10]. On Ethereum, such computation is prohibitively expensive on-chain, creating a structural dependence on this centralized component. Additionally, CoW DAO's governance process enforces social consensus rules around surplus allocation and buffer usage, rather than through code.

This design reflects a fundamental constraint in Ethereum: on-chain computation costs make complex auction logic prohibitively expensive. While Autopilot addresses this need, it introduces a critical centralization point whose failure modes have been demonstrated in practice.

3.1.4 Fair Combinatorial Batch Auction

CoW Protocol's auction mechanism, termed Fair Combinatorial Auction [9], selects winners through combinatorial optimization over solution sets. Solvers can submit individual bids targeting single intents or batched bids covering multiple intents. The Autopilot evaluates these submissions in stages.

First, for each token pair and direction, the Autopilot identifies the best individual bid. This establishes a reference price representing the best achievable execution against external liquidity. Next, it filters solutions that underperform this reference, discarding any that deliver less surplus than the baseline for any token pair. Finally, it selects the combination maximizing total surplus, subject to the constraint that all intents within a directed pair must use the same clearing price.

This mechanism enforces Uniform Directional Clearing Price (UDCP) [10], ensuring that within a single solution, all intents trading the same token pair in the same direction receive the same exchange rate. By decoupling execution prices from transaction ordering, this property prevents MEV sandwich attacks.

3.1.5 Forms of Coincidence of Wants

CoW matching can take several forms, depending on the structure of the trading pair [11].

When two users exchange exactly what the other seeks, a *simple* CoW occurs with no intermediary liquidity needed. This may be complete, with both sides fully matched, or partial, where one side is fully filled while the other accesses external liquidity.

Users selling in the same direction may be aggregated into a single transaction, reducing gas costs through *batching*. Though this form still interacts with AMMs, it materially improves efficiency per intent.

More complex forms include matches at intermediate tokens in multi-hop routes. For example, if user A routes CRV→ETH→USDT and user B routes USDT→ETH→COW, the ETH↔USDT leg can be settled peer-to-peer, requiring solvers capable of multi-step decomposition.

Finally, *ring trades* involve three or more users forming closed exchange cycles, where

user A sells X for Y, user B sells Y for Z, and user C sells Z for X. Finding such cycles requires sophisticated optimization.

3.1.6 The Solver Whitelist Mechanism

CoW Protocol requires solver authorization, though the reasons are often misunderstood. The whitelist is not primarily a mechanism to prevent price manipulation—the smart contract's limit price constraint and EVM revert already handle that. Instead, it serves two operational functions: protecting the centralized Autopilot from DDoS attacks via spam submissions, and ensuring that winning solvers actually execute within the required deadline.

This design introduces a tension: the Autopilot's necessity to reduce on-chain costs requires protecting it from spam through a solver whitelist, which in turn becomes a centralization vector. The 2023 solver exploit illustrates this risk [6]. When a whitelisted solver was compromised, the unlimited ERC-20 approvals previously granted to the vault relayer became an exploitable attack surface, resulting in direct fund losses. This incident demonstrates that architectures relying on privileged intermediaries to authorize asset movement carry systemic trust risks that governance alone cannot fully mitigate.

3.2 CoW Protocol on Ethereum versus the Proposed Design on Sui

3.2.1 Structural Limitations of the Account-Based Model

CoW Protocol's design reflects constraints inherent to Ethereum's account-based model, where all state resides in a single global tree and every modification passes through sequential consensus. User intents exist off-chain as signed messages, with assets remaining in user wallets but authorized via ERC-20 `approve`. EVM revert guarantees atomicity, but winner selection logic must live off-chain due to computation costs.

This architecture produces measurable limitations. All critical auction logic—score computation, winner selection, and UDCP and EBBO verification—depends on a

centralized Autopilot, creating a single point of failure and potential censorship vector. The `approve` mechanism creates a persistent vulnerability: users typically grant unlimited approvals once, so any vault relayer vulnerability can drain entire balances [5]. Gas fee volatility on Ethereum also creates barriers to frequent batching. Although batch windows are 36 seconds, solvers prioritize high-value intents, potentially leaving retail users with inconsistent execution priority.

3.2.2 Design Opportunities of Sui's Object Model

Sui applies an object-centric model where each unit of state exists as an object with a unique identifier and explicit owner. The Move language enforces object integrity through a linear type system: no object can be copied or destroyed arbitrarily, and ownership is tracked at the compiler level.

This model enables three design improvements for CoW on Sui. First, intents can be represented as on-chain shared objects with assets locked directly via escrow, completely eliminating the `approve` pattern and closing the 2023 exploit's attack surface. Second, Sui's low and stable gas fees enable bringing the full auction logic on-chain through Commit-Reveal mechanisms, removing dependence on a centralized Autopilot. Third, stable transaction costs make frequent micro-batching (1–2 second intervals) economically sustainable, ensuring consistent execution regardless of intent size.

CoW Protocol on Ethereum stores intents off-chain as signed messages with assets in user wallets, authorized via ERC-20 `approve`. On-chain, it enforces limit prices and solver whitelists, but the full auction logic including winner selection and EBBO verification [10] occurs off-chain in the centralized Autopilot. The proposed design on Sui instead relocates intents on-chain as shared objects with assets locked directly via escrow, eliminates the approve requirement, brings all auction logic on-chain via Commit-Reveal, and enables permissionless solver participation through smart contract bonds rather than governance whitelists. Settlement atomicity is guaranteed through the Hot Potato pattern rather than EVM revert, and reduced transaction costs enable micro-batching at 1–2 second intervals instead of 36 seconds.

3.3 Proposed Model: On-Chain Intent-Based Batch Auction on Sui

3.3.1 Intent as an On-Chain Shared Object

The core design choice is relocating the intent from an off-chain signed message to an on-chain shared object with locked assets.

```
struct Intent<phantom S, phantom B> has key {
    id: UID,
    owner: address,
    sell_coin: Coin<S>,       // asset locked in the object at creation time
    min_amount_out: u64,      // limit price: minimum tokens of type B to rec
    deadline: u64,            // validity epoch
    batch_id: Option<ID>,     // None if pending; Some if assigned to a batch
}
```

The distinction from CoW Protocol's approach lies in asset custody handling. Ethereum's account-based model requires users to grant ERC-20 `approve` to an intermediary contract so solvers can move assets—an authorization mechanism creating persistent attack surface. Sui's object-centric model has no equivalent approval concept. Assets are transferred entirely into the intent object at creation time through an on-chain escrow pattern. The `sell_coin` is embedded directly within the Intent and governed entirely by Move's ownership system. No external address—including winning solvers—can access this asset except through contract-defined functions. The attack surface that enabled the 2023 exploit does not exist in this architecture.

3.3.2 On-Chain Batch Auction Lifecycle

The system operates through a continuous batch cycle with all phases enforced on-chain.

During **intent collection**, users submit intents at any time. The intent pool remains continuously available without batch-window constraints on submission.

In the **batch opening** phase, a keeper periodically calls `open_batch`, locking the

current pending set into a new batch and transitioning it to Commit phase. No new intents can be added after this point.

During the **commit phase**, solvers compute solutions off-chain and submit `commit_hash = hash(solution || nonce)` with an on-chain bond. Only a cryptographic hash is published, preventing solvers from extracting information about competitors' solutions from the chain. This provides confidentiality equivalent to CoW Protocol's off-chain Autopilot, without requiring a centralized coordinator.

In the **reveal and winner selection** phase, after the commit window closes, each solver reveals their `nonce` to verify the previously submitted hash. The contract reads the `actual_surplus` for each solution directly from on-chain batch state and selects the solver with the highest `total_surplus` as the winner. The full solution is not re-submitted; all scoring information was recorded on-chain during the commit phase.

During the **execute phase**, the winning solver constructs and executes the Programmable Transaction Block containing settlement logic. The winning solver's bond remains locked throughout this phase and is released only after `close_settlement` confirms that `actual_surplus` meets the committed threshold. Losing solvers' bonds are returned after a fixed waiting period without requiring any action.

In the final **done or aborted** phase, the batch concludes. Any intents not processed are released back to the pending pool to participate in the next batch.

3.4 On-Chain Solver Selection via Commit-Reveal

3.4.1 Why Commit-Reveal Rather Than Open Bid

On Ethereum, CoW Protocol operates with an open bid model because the auction is conducted off-chain and opaque to all participants—solvers submit solutions to Autopilot and cannot observe each other's proposals before the deadline. When the entire auction process moves on-chain on Sui, this condition changes: every transaction becomes publicly observable. If solver A submits a solution directly on-chain, solver B could read it from the mempool, copy it, and submit an identical transaction with a higher gas fee to be prioritized ahead—a classic solution front-running attack.

Commit-Reveal addresses this by separating the auction into two temporally distinct phases. In the commit phase, solvers publish only the hash of their solution, revealing no exploitable information. Only after all commits are recorded does the reveal phase begin, at which point solution contents become visible. This approach replicates the confidentiality properties of CoW Protocol's off-chain auction on-chain, without delegating trust to a centralized coordinator.

3.4.2 Game-Theoretic Analysis of the Bond Mechanism

The solver bond serves multiple purposes beyond anti-spam functionality. It establishes a Nash equilibrium in the solver competition. Let V denote the reward for the winning solver, B the bond value, and p the probability that a given solver wins. The rational participation condition is [12]:

$$p \cdot V - (1-p) \cdot B > 0 \implies p > \frac{B}{V+B}$$

With B calibrated appropriately, only solvers with sufficiently high confidence in their solution quality will participate. Two principal attack strategies are thereby eliminated: *griefing*, where a solver submits a commit without a valid solution, is deterred because the bond is forfeited if the solver fails to reveal on time; and *score inflation*, where a solver overstates the committed score and then executes below that level, is deterred by the `EScoreMismatch` penalty enforced on-chain.

Most significantly, this mechanism enables permissionlessness: any party may participate as a solver without undergoing a DAO whitelist process. The bond substitutes for institutional trust—rather than relying on reputation and governance, the system relies on on-chain locked capital.

3.4.3 On-Chain Penalty Conditions

The contract enforces two penalty conditions that protect system integrity.

EScoreMismatch is triggered when `actual_surplus < committed_score × 95%`, upon which the solver's bond is forfeited. The 95% threshold provides a buffer for small price movements between computation and execution, while remaining tight enough to deter material score inflation.

EDeadlineExpired is triggered when the winning solver fails to execute settlement within the prescribed window after being selected, upon which the bond is also forfeited. This condition eliminates the "win and abandon" risk that CoW Protocol on Ethereum addresses through the solver whitelist, replacing an institutional guarantee with an economic one.

3.5 Type-Level Atomicity in Move

3.5.1 EVM Revert versus Type-Level Correctness

In the EVM, settlement atomicity is guaranteed by the revert mechanism: if any step in settlement fails, the entire transaction is rolled back. This is a necessary but insufficient guarantee. It prevents incorrect settlement but does not prevent *intentional partial settlement*, where a solver deliberately skips unprofitable intents while processing the remainder. If the contract does not explicitly check every intent in the batch, no revert will occur. The correctness of the system thus depends on the completeness of runtime validation logic—a significant attack surface in any complex multi-step contract.

3.5.2 The Hot Potato Pattern in Move

Move's linear type system provides a structurally stronger guarantee. A *hot potato* is a struct that has neither the `drop` ability nor the `store` ability [13]. As a direct consequence, a hot potato value, once created, *must be consumed* by a designated function within the same transaction. It cannot be discarded, and it cannot be saved to storage. This is a compiler-enforced constraint, not a runtime check.

`SettlementTicket` is designed as a hot potato:

```
struct SettlementTicket {   // no `drop`, no `store`
    batch_id: ID,
    intents_to_process: vector<ID>,  // must be fully emptied before closing
    actual_surplus: u64,
}
```

The execution flow is strictly enforced by the type system:

```
open_settlement(state, clock) → SettlementTicket    // created; cannot be dro
      ↓
process_intent<S,B>(ticket, intent, coin, ...)      // removes one intent fro
      ↓  [repeated for each intent in the batch]
close_settlement(state, ticket, config)             // consumes the ticket; v
```

If the solver does not call `close_settlement`, the `SettlementTicket` remains live at the end of the transaction and the Move VM rejects it. If the solver calls `close_settlement` while `intents_to_process` is non-empty, the contract aborts. There is no escape path: partial settlement is not merely penalized—it is structurally impossible. This is *type-level correctness*, elevating the guarantee from a runtime concern to a compile-time impossibility.

3.6 Matching Algorithm and Surplus Theory

3.6.1 Surplus Definition and Winner Selection Metric

Consistent with CoW Protocol's definition, the surplus of an intent i is the positive difference between the actual amount of tokens received r_i and the user's minimum acceptable amount m_i:

$$\text{surplus}_i = r_i - m_i \quad \text{subject to } r_i \geq m_i$$

The total surplus of a batch serves as the on-chain metric for comparing solver solutions and selecting the winner:

$$\text{Score} = \sum_{i \in \text{batch}} (r_i - m_i)$$

This is consistent with CoW Protocol's principle that the solver wins precisely when users win. The proposed design applies *individual pricing*, where each intent receives an execution price based on settlement conditions, rather than CoW Protocol's Uniform Directional Clearing Price, which requires all intents within a directed pair to receive the same rate. Individual pricing offers greater optimization flexibility and is appropriate for Sui's high-frequency, small-batch environment, where enforcing UDCP

across a few intents would provide limited fairness benefit relative to the added complexity.

3.6.2 CoW Validity Conditions

Two intents A and B form a valid Simple CoW if and only if:

$$A.\text{sellType} = B.\text{buyType} \quad \wedge \quad B.\text{sellType} = A.\text{buyType}$$

$$A.\text{sellAmount} \geq B.\text{minAmountOut} \quad \wedge \quad B.\text{sellAmount} \geq A.\text{minAmountOut}$$

In the case of a Partial CoW, only one directional condition needs to be satisfied. The side that is fully matched receives its complete fill through the peer-to-peer exchange; the remaining portion of the other side is routed through DeepBook as an AMM fallback.

3.6.3 On-Chain Price Floor as a Substitute for EBBO

CoW Protocol on Ethereum guarantees EBBO—Ethereum Best Bid and Offer, representing the best available price on on-chain AMMs—through Autopilot verification conducted off-chain [10]. In the proposed design, an equivalent guarantee is achieved through *on-chain price verification* performed within the settlement transaction itself. At the moment of execution, the contract reads `mid_price` directly from DeepBook and enforces:

$$\text{price_floor}_i = \max\left(m_i, \text{mid_price} \times 99\%\right)$$

If `actual_payout < price_floor`, the contract aborts the transaction immediately. This requires no external oracle, no off-chain infrastructure, and no trusted third party. The 1% buffer accommodates minor price movements between the commit and execute phases—a very short interval on Sui, but sufficient to introduce small deviations in the reference price.

References

[1] Mysten Lab. *Sui Q2 2025 DeFi Roundup*. 2025. Available at: https://blog.sui.io/q2-2025-defi-roundup/. Accessed: 2026-03-13.

[2] Uniswap. *What Is Maximal Extractable Value (MEV)?* Uniswap Labs. Available at: https://blog.uniswap.org/maximal-extractable-value-mev. Accessed: 2026-03-13.

[3] Dune. *Batch History | Dune*. 2023. Available at: https://www.dune.com/cowprotocol/batch-history. Accessed: 2026-03-13.

[4] Mysten Lab. *MEV on Sui: Current State and Next Steps*. 2025. Available at: https://blog.sui.io/mev-on-sui-current-state/. Accessed: 2026-03-13.

[5] Decrypt / Sebastian Sinclair. *Crypto Trader Loses Nearly $50M in Aave Trade, Protocol Offers $600K Fee Refund*. Decrypt. 2026. Available at: https://decrypt.co/360961/crypto-trader-loses-nearly-50m-aave-trade-600k-fee-refund. Accessed: 2026-03-13.

[6] CoW DAO. *CoW Swap Solver Exploit— Post Mortem 07/02/2023 - CoW DAO*. Available at: https://cow.fi/learn/cow-swap-solver-exploit-post-mortem. Accessed: 2026-03-13.

[7] Cow DAO. *Documentation - CoW DAO*. Available at: https://docs.cow.fi/. Accessed: 2026-03-13.

[8] Ethereum Foundation. *ERC-20 Token Standard*. ethereum.org. Available at: https://ethereum.org/developers/docs/standards/tokens/erc-20/. Accessed: 2026-03-13.

[9] CoW DAO. *Fair Combinatorial Batch Auction - CoW Protocol*. Available at: https://docs.cow.fi/cow-protocol/concepts/introduction/fair-combinatorial-auction. Accessed: 2026-03-13.

[10] CoW DAO. *Competition Rules - CoW Protocol*. Available at: https://docs.cow.fi/cow-protocol/reference/core/auctions/competition-rules. Accessed: 2026-03-13.

[11] “A Coincidence of Wants Mechanism for Swap Trade Execution in Decentralized Exchanges”Abhimanyu Nag, Madhur Prabhakar, and Tanuj Behl. arXiv.org. 2025. Available at: https://arxiv.org/abs/2507.10149v1. Accessed: 2026-02-20.

[12] “Non-Cooperative Games”John Nash. , Vol. 54Annals of Mathematics. , pp. 286–2951951. Available at: https://www.jstor.org/stable/1969529. Accessed: 2026-03-13.

[13] Move Book. *Pattern: Hot Potato | The Move Book*. Available at: https://move-book.com/programmability/hot-potato-pattern/. Accessed: 2026-03-13.

[14] “An Analysis of Intent-Based Markets”Tarun Chitra, Kshitij Kulkarni, Mallesh Pai, and Theo Diamandis. arXiv.org. 2024. Available at: https://arxiv.org/abs/2403.02525v2. Accessed: 2026-02-20.

Appendix A

This is title of appendix A

Appendix B

This is title of appendix B

Temporary page!

LaTeX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because LaTeX now knows how many pages to expect for this document.